Python网络爬虫与信息提取

(MOOC 北京理工大学)

(笔记)

(开始时间：2021/3/25)

(by 孟智超)

1 CATLOG

1 CATLOG
2 第一章、网络爬虫之规则
2.1 案例1：requests_时间测试
2.2 案例2：requests保存图片
2.3 案例3：requests保存gif动画
2.4 案例4：requests保存视频（MP4格式）
2.5 案例5：requests保存html
2.6 案例6：requests_百度搜索关键词提交
2.7 案例7：requests_360搜索关键词提交
2.8 案例8：requests_电话号码归属地查询
2.9 requests_ip地址归属地查询
3 第二章、网络爬虫之提取
3.1 中国大学排名爬取(requests+json)
3.2 案例9：中国大学排名定向爬取（教学提供，网址失效）
3.3 案例10：大学排名top100（bs4+requests）
4 第三章、网络爬虫之实战
4.1 正则表达式
4.2 正则表达式总结
4.3 python正则表达式库（re）
4.4 案例11、股票信息爬取
5 第四章 网络爬虫之框架
5.1 scrapy框架的简介
5.2 scrapy框架的数据类型
5.3 scrapy爬虫提取信息的方法
5.4 scrapy小例子：爬取网页html并保存

2 第一章、网络爬虫之规则

2.1 案例1：requests_时间测试

```python
# -*- coding: utf-8 -*-

"""
**************************************************
@author:   Ying
@software: PyCharm
@file: 01、request_时间测试.py
@time: 2021/3/26 12:34
**************************************************
"""

import requests
import time


# import warnings
#
# warnings.filterwarnings('ignore')


def get_html(url):
    headers = {'User-Agent': 'Chrome/89'}
    try:
        r = requests.get(url, headers=headers)
        print(r.encoding)
        print(r.apparent_encoding)
        print("状态码：{}".format(r.status_code))
        print('头部信息：{}'.format(r.request.headers))
        r.raise_for_status()
    except:
        print('抓取失败')


def main():
    start_time = time.time()
    url = 'https://www.baidu.com/'
    for i in range(10):
        res = get_html(url)
    end_time = time.time()
    print('10次共用时:{}'.format(end_time - start_time))


if __name__ == '__main__':
    main()
```

2.2 案例2：requests保存图片

```python
# -*- coding: utf-8 -*-

"""
**************************************************
@author:   Ying
@software: PyCharm
@file: 02、requests_保存图片、视屏、gif、html.py

@time: 2021/3/26 12:35
**************************************************
"""

import requests

'''图片'''
try:
    # url为所需要爬取图片的网址
    url =
'https://gimg2.baidu.com/image_search/src=http%3A%2F%2Fattach.bbs.miui.com%2Fforum%2F201303%2F16%2F173710lvx470i4348z6i6z.jpg&refer=http%3A%2F%2Fattach.bbs.miui.com&app=2002&size=f9999,10000&q=a80&n=0&g=0n&fmt=jpeg?sec=1619313958&t=c8be426e46c92938a80afae57b03aa8c'
    headers = {'User-Agent': 'Chrome/89'}
    r = requests.get(url, headers=headers)
    r.encoding = r.apparent_encoding
    print('长度: {} KB'.format(len(r.text) / 1024))

    # r.raise_for_status()
    file_path = './save/pic_1.jpg'
    with open(file_path, 'wb') as f:  # 注意: wb 是以二进制文件写入
        f.write(r.content)  # content 是二进制文件

except:
    print('抓取失败')
```

2.3 案例3：requests保存gif动画

```python
# -*- coding: utf-8 -*-

"""
**************************************************
@author:   Ying
@software: PyCharm
```

```
@file: 02、requests_保存图片、视屏、gif、html.py

@time: 2021/3/26 12:35
**************************************************
"""

import requests

"""gif"""
try:
    url = 'https://gimg2.baidu.com/image_search/src=http%3A%2F%2F5b0988e595225.cdn.sohucs.com%2Fq_70%2Cc_zoom%2Cw_640%2Fimages%2F20180930%2F2369f497741e4f65a89ab70903599ac5.gif&refer=http%3A%2F%2F5b0988e595225.cdn.sohucs.com&app=2002&size=f9999,10000&q=a80&n=0&g=0n&fmt=jpeg?sec=1619314331&t=94eb3c01580846e8083ac24c1198e829'
    headers = {'User-Agent': 'Chrome/89'}
    r = requests.get(url, headers=headers)
    r.encoding = r.apparent_encoding
    print('长度: {} KB'.format(len(r.text) / 1024))

    r.raise_for_status()
    file_path = './save/gif_1.gif'
    with open(file_path, 'wb') as f:  # 注意: wb 是以二进制文件写入
        f.write(r.content)  # content是二进制文件

except:
    print('抓取失败')
```

2.4 案例4：requests保存视频（MP4格式）

```
# -*- coding: utf-8 -*-

"""
**************************************************
@author:   Ying
@software: PyCharm
@file: 02、requests_保存图片、视屏、gif、html.py

@time: 2021/3/26 12:35
**************************************************
"""

import requests

"""保存视屏"""
try:
```

```
    url = 'https://vd2.bdstatic.com/mda-mcqdktmhq2x9eicx/sc/cae_h264_clips/1616636519/mda-mcqdktmhq2x9eicx.mp4?auth_key=1616724555-0-0-5cccc0a05823821cf587e7c9ee878a95&bcevod_channel=searchbox_feed&pd=1&pt=3&abtest='
    headers = {'User-Agent': 'Chrome/89'}
    r = requests.get(url, headers=headers)
    r.encoding = r.apparent_encoding  # encoding 是从 html head中获取的字符集
    print('长度: {} KB'.format(len(r.text) / 1024))

    r.raise_for_status()
    file_path = './save/抵制新疆棉花.mp4'
    with open(file_path, 'wb') as f:  # 注意: wb 是以二进制文件写入
        f.write(r.content)  # content是二进制文件

except:
    print('抓取失败')
```

2.5 案例5：requests保存html

```
# -*- coding: utf-8 -*-

"""
**************************************************
@author:   Ying
@software: PyCharm
@file: 02、requests_保存图片、视屏、gif、html.py

@time: 2021/3/26 12:35
**************************************************
"""

import requests

"""保存html"""

url = 'https://www.amazon.cn/gp/product/B01M8L5z3Y'
try:
    headers = {'User-Agent': 'Chrome/89'}
    r = requests.get(url, headers=headers)
    r.encoding = r.apparent_encoding
    print('长度: {} KB'.format(len(r.text) / 1024))
    r.raise_for_status()
    file_path = './save/亚马逊中国主页.html'
    with open(file_path, 'w+') as f:  # 这里不可以用 wb,  wb是以二进制写入
        f.write(r.text)
        f.close()

except:
```

```
    print('抓取失败')
```

2.6 案例6：requests_百度搜索关键词提交

```
# -*- coding: utf-8 -*-

"""
**************************************************
@author:   Ying
@software: PyCharm
@file: 03、requests_百度-306搜索关键词提交.py
@time: 2021/3/26 12:37
**************************************************
"""
"""
百度 搜索引擎关键词提交接口
百度：http://www.baidu.com/s?wd=keyword
"""

import requests

"""百度搜索引擎关键词提交"""
keyword = 'Python'
try:
    kv = {'wd': keyword}
    r = requests.get('http://www.baidu.com/s', params=kv)
    print(r.request.url)
    r.raise_for_status()
    print('{}KB'.format(len(r.text)/1024))

    file_path = './save/百度关键词提交.html'
    with open(file_path, 'w+', encoding='utf-8') as f:
        f.write(r.text)

except:
    print('爬取失败')
```

2.7 案例7：requests_360搜索关键词提交

```
# -*- coding: utf-8 -*-

"""
**************************************************
@author:   Ying
@software: PyCharm
@file: 03、requests_百度-306搜索关键词提交.py
```

```
@time: 2021/3/26 12:37
**************************************************
"""
"""
360 搜索引擎关键词提交接口
306:  http://www.so.com/s?q=keyword
"""

import requests

"""360搜索引擎关键词提交"""
keyword = 'Python'
try:
    kv = {'q': keyword}
    r = requests.get('http://www.so.com/s', params=kv)
    print(r.request.url)
    r.raise_for_status()
    print('{}KB'.format(len(r.text)/1024))

    file_path = './save/360关键词提交.html'
    with open(file_path, 'w+', encoding='utf-8') as f:
        f.write(r.text)
        f.close()

except:
    print('爬取失败')
```

2.8 案例8：requests_电话号码归属地查询

```
# -*- coding: utf-8 -*-

"""
**************************************************
@author:   Ying
@software: PyCharm
@file: 04、requests_电话号码归属地.py
@time: 2021/3/26 12:38
**************************************************
"""

"""
手机号码自动归属地查询提交接口
url: https://m.ip138.com/sj.asp?mobile=15375376068

"""

import requests
from lxml import etree as ee
import csv
```

```python
import pandas as pd

"""电话号码"""


def get_html(url, phone_num):
    try:
        headers = {'User-Agent': 'Chrome/89'}
        kv = {'mobile': phone_num}
        r = requests.get(url, headers=headers, params=kv)

        r.raise_for_status()  # 如果发生异常则抛出
        html = r.content.decode(r.apparent_encoding)
        html = ee.HTML(html)
        data_list = html.xpath('//div[@class="module mod-panel"]/div[@class="bd"]/table/tbody//tr')
        phone_address_info = []
        for data in data_list:
            res = data.xpath('./td/span/text()')[0]
            phone_address_info.append(res)
        print(phone_address_info)

        save_1(phone_address_info)

    except:
        print("抓取失败")


def save_1(li):
    with open('./save/号码归属地.csv', 'a') as f:
        w = csv.writer(f)
        w.writerow(li)


def main():
    data = pd.read_excel('./need/电话簿.xlsx',sheet_name='Sheet2')
    data.drop_duplicates(inplace=True)
    phone_list = data['tel_phone'].values.tolist()

    with open('./save/号码归属地.csv', 'w') as f:
        w = csv.writer(f)
        w.writerow(['手机号码段', '卡号归属地', '卡 类 型', '区 号', '邮 编'])

    for phone_num in phone_list:
        url = 'https://m.ip138.com/sj.asp'
        get_html(url, phone_num)


if __name__ == "__main__":
    main()
```

2.9 requests_ip地址归属地查询

```python
# -*- coding: utf-8 -*-

"""
**************************************************
@author:   Ying
@software: PyCharm
@file: 05、requests_ip地址查询.py
@time: 2021/3/26 12:38
**************************************************
"""

"""
ip 地址 自动归属地查询提交接口
url: https://m.ip138.com/ip.asp?ip=183.160.212.66
"""

import requests
from lxml import etree as ee

try:
    url = 'url: https://m.ip138.com/ip.asp'
    headers = {'User-Agent': 'Chrome/89'}
    ip_address = '183.160.212.66'
    kv = {'ip': ip_address}
    r = requests.get(url, headers=headers, params=kv)

    r.raise_for_status()

    print(r.text[-550:])
except:
    print('抓取失败')
```

3 第二章、网络爬虫之提取

3.1 中国大学排名爬取(requests+json)

```python
# -*- coding: utf-8 -*-

"""
**************************************************
@author:   Ying
@software: PyCharm
@file: 02、2020年中国大学排名.py
@time: 2021-04-08 13:20
**************************************************
"""

import requests
import pandas as pd
# import bs4
# from bs4 import BeautifulSoup
import json
import csv

csv_flag = True


def get_html(url, headers):
    try:
        html = requests.get(url, headers)
        html.encoding = html.apparent_encoding
        print('状态码:{}'.format(html.status_code))
        html.raise_for_status()
        return html.content.decode()
    except:
        print('抓取失败')


def save_as_csv(info_list):
    """保存方法1——通过文件写入"""
    global csv_flag
    with open('./save/中国大学2020排名_1.csv', 'a') as f:
        w = csv.writer(f)
        if csv_flag:
            w.writerow(['排名', '大学中文名称', '大学英文名称',
                        '所在省份', '办学类型', '总分', '大学标签',
                         '办学层次得分', '学科水平得分', '办学资源得分',
                        '师资规模与结构得分', '人才培养得分',
                         '科学研究得分', '服务社会得分', '高端人才得分',
                        '重大项目与成果得分', '国际竞争力得分'])
            csv_flag = False
```

```python
        else:
            w.writerow(info_list)


def main():
    url = 'https://www.shanghairanking.cn/api/pub/v1/bcur?bcur_type=11&year=2020'
    headers = {'User-Agent': 'Chrome/89'}
    html = get_html(url, headers)
    datas = json.loads(html)
    data = datas['data']
    rankings_info = data['rankings']
    university_info = []
    for ranking in rankings_info:
        rank = ranking['ranking']  # 排名
        univNameCn = ranking['univNameCn']  # 大学中文名称
        univNameEn = ranking['univNameEn']  # 大学英文名称
        province = ranking['province']  # 所在省份
        univCategory = ranking['univCategory']  # 办学类型
        score = ranking['score']  # 总分
        univTags = ranking['univTags']  # 大学标签
        score_59 = ranking['indData']['59']  # 办学层次得分
        score_60 = ranking['indData']['60']  # 学科水平得分
        score_61 = ranking['indData']['61']  # 办学资源得分
        score_62 = ranking['indData']['62']  # 师资规模与结构得分
        score_63 = ranking['indData']['63']  # 人才培养得分
        score_64 = ranking['indData']['64']  # 科学研究得分
        score_65 = ranking['indData']['65']  # 服务社会得分
        score_66 = ranking['indData']['66']  # 高端人才得分
        score_67 = ranking['indData']['67']  # 重大项目与成果得分
        score_68 = ranking['indData']['68']  # 国际竞争力得分

        info = [rank, univNameCn, univNameEn, province,
                univCategory, score, univTags, score_59,
                score_60, score_61, score_62, score_63,
                score_64, score_65, score_66, score_67, score_68]

        save_as_csv(info)

        university_info.append(info)

    univer_data = pd.DataFrame(university_info)
    univer_data.columns = ['排名', '大学中文名称', '大学英文名称',
                           '所在省份', '办学类型', '总分', '大学标签',
                           '办学层次得分', '学科水平得分', '办学资源得分',
                           '师资规模与结构得分', '人才培养得分',
                           '科学研究得分', '服务社会得分', '高端人才得分',
                           '重大项目与成果得分', '国际竞争力得分']

    # 保存方法2 通过pandas的DataFrame直接保存为csv，当然还可以保存为excel等
    univer_data.to_csv('./save/中国大学2020排名_2.csv')
    univer_data.to_excel('./save/中国大学2020排名_2.xlsx')
```

```python


if __name__ == '__main__':
    main()



"""
2020年大学排名网站：https://www.shanghairanking.cn/rankings/bcur/2020
该网址实际展示已经经过js渲染后展示出，所以上述网址返回的网页源代码中并没有我们看到的大学排名信息

返回信息url：https://www.shanghairanking.cn/api/pub/v1/bcur?bcur_type=11&year=2020
返回数据类型：json
"""
```

3.2 案例9：中国大学排名定向爬取（教学提供，网址失效）

```python
# -*- coding: utf-8 -*-

"""
**************************************************
@author:   Ying
@software: PyCharm
@file: 01、BeautifulSoup_中国大学排名定向爬虫.py
@time: 2021/3/31 19:25
**************************************************
"""

import requests
from bs4 import BeautifulSoup
import bs4


def getHTMLText(url):
    try:
        r = requests.get(url, timeout=30)
        r.raise_for_status()
        r.encoding = r.apparent_encoding
        return r.text
    except:
        return ""


def fillUnivList(ulist, html):
    soup = BeautifulSoup(html, "html.parser")
```

```
    for tr in soup.find('tbody').children:
        if isinstance(tr, bs4.element.Tag):
            tds = tr('td')
            ulist.append([tds[0].string, tds[1].string, tds[3].string])


def printUnivList(ulist, num):
    print("{:^10}\t{:^6}\t{:^10}".format("排名", "学校名称", "总分"))
    for i in range(num):
        u = ulist[i]
        print("{:^10}\t{:^6}\t{:^10}".format(u[0], u[1], u[2]))


def main():
    uinfo = []
    url = 'https://www.zuihaodaxue.cn/zuihaodaxuepaiming2016.html'
    html = getHTMLText(url)
    fillUnivList(uinfo, html)
    printUnivList(uinfo, 20)  # 20 univs


if __name__ == '__mian__':
    main()

```

3.3 案例10：大学排名top100（bs4+requests）

```
# -*- coding: utf-8 -*-

"""
**************************************************
@author:   Ying
@software: PyCharm
@file: 03、大学排名top100.py
@time: 2021-04-08 14:50
**************************************************
"""
import requests
import numpy as np
import pandas as pd
import bs4
from bs4 import BeautifulSoup
import csv


def getHtml_text(url):
    try:
        headers = {'User-Agent': 'Chrome/89'}
        res = requests.get(url, headers)
```

```python
        res.raise_for_status()
        res.encoding = res.apparent_encoding
        return res.content.decode()

    except:
        return ''


def save_as_csv(infoList):
    """
    第一种方法保存为csv文件
    :param infoList:
    :return:
    """
    with open('./save/中国大学top100_1.csv', 'a') as f:
        w = csv.writer(f)
        w.writerow(infoList)


def main():
    url = """http://www.cnur.com/rankings/188.html"""
    demo = getHtml_text(url)

    soup = BeautifulSoup(demo, 'html.parser')

    university_info = []
    for tr in soup.find('tbody'):
        if isinstance(tr, bs4.element.Tag):
            tds = tr('td')
            info = [tds[0].string, tds[1].string, tds[2].string, tds[3].string]

            save_as_csv(info)

            university_info.append(info)

    # 第二种方法保存为csv文件
    data = pd.DataFrame(university_info)
    data.columns = data.iloc[0, :]  # 将第一行改为指标名称
    data = data.iloc[1:, :]  # 将第一行过滤
    data.to_csv('./save/中国大学top100_2.csv', index=None)


if __name__ == '__main__':
    main()

```

4 第三章、网络爬虫之实战

4.1 正则表达式

- 正则表达式常用操作符：

操作符	说明	实例
.	表示任何单个字符	
[ ]	字符集，对单个字符给出取值范围	[abc]表示a、b、c，[a-z]表示a到z单个字符
[^]	非字符集，对单个字符给出排除范围	[^abc]表示非a或b或c的单个字符
*	前一个字符0次或无限次扩展	abc* 表示 ab、abc、abcc、abccc等
+	前一个字符1次或无限次扩展	abc+ 表示 abc、abcc、abccc等
?	前一个字符0次或1次扩展	abc? 表示 ab、abc
|	左右表达式任意一个	abc|def 表示 abc、def

操作符	说明	实例
{m}	扩展前一个字符m次	ab{2}c表示abbc
{m,n}	扩展前一个字符m至n次（含n）	ab{1,2}c表示abc、abbc
^	匹配字符串开头	^abc表示abc且在一个字符串的开头
$	匹配字符串结尾	abc$表示abc且在一个字符串的结尾
( )	分组标记，内部只能使用 | 操作符	(abc)表示abc，(abc|def)表示abc、def
\d	数字，等价于[0-9]	
\w	单词字符，等价于[A-Za-z0-9_]	

- 正则表达式实例：

正则表达式	对应字符串
P(Y|YT|YTH|YTHO)?N	'PN'、'PYN'、'PYTN'、'PYTHN'、'PYTHON'
PYTHON+	'PYTHON'、'PYTHONN'、'PYTHONNN' …
PY[TH]ON	'PYTON'、'PYHON'
PY[^TH]?ON	'PYON'、'PYaON'、'PYbON'、'PYcON'…
PY{:3}N	'PN'、'PYN'、'PYYN'、'PYYYN'…

- 经典正则表达式 实例：

^[A-Za-z]+$	由26个字母组成的字符串
^[A-Za-z0-9]+$	由26个字母和数字组成的字符串
^-?\d+$	整数形式的字符串
^[0-9]*[1-9][0-9]*$	正整数形式的字符串
[1-9]\d{5}	中国境内邮政编码，6位
[\u4e00-\u9fa5]	匹配中文字符
\d{3}-\d{8}|\d{4}-\d{7}	国内电话号码，010-68913536

- 匹配IP地址 的正则表达式：

IP地址字符串形式的正则表达式（IP地址分4段，每段0-255）

\d+.\d+.\d+.\d+ 或 \d{1,3}.\d{1,3}.\d{1,3}.\d{1,3}

精确写法 0-99： [1-9]?\d

100-199: 1\d{2}

200-249: 2[0-4]\d

250-255: 25[0-5]

```
(([1-9]?\d|1\d{2}|2[0-4]\d|25[0-5]).){3}([1-9]?\d|1\d{2}|2[0-4]\d|25[0-5])
```

4.2 正则表达式总结

- 多种匹配模式

实例	描述
`[Pp]ython`	匹配 “Python” 或 “python”。
`rub[ye]`	匹配 “ruby” 或 “rube”。
`[abcdef]`	匹配中括号内的任意一个字母。
`[0-9]`	匹配任何数字。类似于 [0123456789]。
`[a-z]`	匹配任何小写字母。
`[A-Z]`	匹配任何大写字母。
`[a-zA-Z0-9]`	匹配任何字母及数字。
`[^au]`	除了au字母以外的所有字符。
`[^0-9]`	匹配除了数字外的字符。

- 实例

实例	描述
`.`	匹配除 “\n” 之外的任何单个字符。要匹配包括 ‘\n’ 在内的任何字符，请使用象 ‘[.\n]’ 的模式。
`?`	匹配一个字符零次或一次，另一个作用是非贪婪模式
`+`	匹配1次或多次
`*`	匹配0次或多次
`\b`	匹配一个长度为 `0` 的子串
`\d`	匹配一个数字字符。等价于 [0-9]。
`\D`	匹配一个非数字字符。等价于 [^0-9]。
`\s`	匹配任何空白字符，包括空格、制表符、换页符等等。等价于 [\f\n\r\t\v]。
`\S`	匹配任何非空白字符。等价于 [^ \f\n\r\t\v]。
`\w`	匹配包括下划线的任何单词字符。等价于’[A-Za-z0-9_]’。
`\W`	匹配任何非单词字符。等价于 ‘[^A-Za-z0-9_]’。
`\b`	匹配一个长度为 `0` 的子串

4.3 python正则表达式库（re）

- 原生字符串类型：

raw string类型（原生字符串类型）

re库采用raw string类型表示正则表达式，表示为：

r'text'

例如：

```
r'[1-9]\d{5}'
r'\d{3}-\d{8}|\d{4}-\d{7}'
```

raw string是不包含对转义符再次转义的字符串

- re库 主要功能函数：

函数	说明
re.search()	在一个字符串中搜索匹配正则表达式的第一个位置，返回match对象
re.match()	从一个字符串的开始位置起匹配正则表达式，返回match对象
re.findall()	搜索字符串，以列表类型返回全部能匹配的子串
re.split()	将一个字符串按照正则表达式匹配结果进行分割，返回列表类型
re.finditer()	搜索字符串，返回一个匹配结果的迭代类型，每个迭代元素是match对象
re.sub()	在一个字符串中替换所有匹配正则表达式的子串，返回替换后的字符串

- re库的面向对象写法：

```
>>> rst = re.search(r'[1-9]\d{5}', 'BIT 100081')
```

函数式用法：一次性操作

⇕

```
>>> pat = re.compile(r'[1-9]\d{5}')
>>> rst = pat.search('BIT 100081')
```

面向对象用法：编译后的多次操作

- `match` 对象介绍

Match对象是一次匹配的结果，包含匹配的很多信息

```
>>> match = re.search(r'[1-9]\d{5}', 'BIT 100081')
>>> if match:
        print(match.group(0))
>>> type(match)
<class '_sre.SRE_Match'>
```

- `match` 对象 `属性`：

属性	说明
.string	待匹配的文本
.re	匹配时使用的patter对象（正则表达式）
.pos	正则表达式搜索文本的开始位置
.endpos	正则表达式搜索文本的结束位置

- `match` 对象 `方法`：

方法	说明
.group(0)	获得匹配后的字符串
.start()	匹配字符串在原始字符串的开始位置
.end()	匹配字符串在原始字符串的结束位置
.span()	返回(.start(), .end())

- match对象 实例 ：

```
>>> import re
>>> m = re.search(r'[1-9]\d{5}', "BIT100081 TSU100084")
>>> m.string
'BIT100081 TSU100084'
>>> m.re
re.compile('[1-9]\\d{5}')
>>> m.pos
0
>>> m.endpos
19
>>> m.group(0)
'100081'
>>> m.start()
3
>>> m.end()
9
>>> m.span()
(3, 9)
```

- re库的 贪婪匹配 和 最小匹配 ：

贪婪匹配

```
>>> match = re.search(r'PY.*N', 'PYANBNCNDN')
>>> match.group(0)
'PYANBNCNDN'
```

Re库默认采用贪婪匹配，即输出匹配最长的子串

最小匹配

如何输出最短的子串呢？

```
>>> match = re.search(r'PY.*?N', 'PYANBNCNDN')
>>> match.group(0)
'PYAN'
```

最小匹配操作符

操作符	说明
*?	前一个字符0次或无限次扩展，最小匹配
+?	前一个字符1次或无限次扩展，最小匹配
??	前一个字符0次或1次扩展，最小匹配
{m,n}?	扩展前一个字符m至n次（含n），最小匹配

只要长度输出可能不同的，都可以通过在操作符后增加?变成最小匹配

- re库总结：

正则表达式是用来简洁表达一组字符串的表达式

```
r'\d{3}-\d{8}|\d{4}-\d{7}'
```

re.search()				regex.search()
re.match()				regex.match()
re.findall()	=	regex=re.compile()	+	regex.findall()
re.split()				regex.split()
re.finditer()				regex.finditer()
re.sub()				regex.sub()

4.4 案例11、股票信息爬取

- 东方财富网>行情中心>沪深个股中 所有板块股票信息爬取
- 网址： http://quote.eastmoney.com/center/gridlist.html#hs_a_board

```python
# -*- coding: utf-8 -*-

"""
**************************************************
@author:   Ying
@software: PyCharm
@file: 01、股票.py
@time: 2021-04-09 16:17
**************************************************
"""

import requests
import traceback
from bs4 import BeautifulSoup
import re
import json
import pandas as pd
import time
import csv
from tqdm import tqdm

flag = True
cur_time = time.strftime('%Y-%m-%d')  # 当前日期（年-月-日）


def get_html(url):
    """
    获取解码后的html文件
```

```python
    :param url:需要解析的网页的数据
    :return:返回解码后的html文件
    """
    try:
        headers = {'User-Agent': 'Chrome/89'}  # 头部信息，模拟Chrome浏览器像服务器发动请求
        r = requests.get(url, headers)  # get方法
        r.raise_for_status()  # 如果出错，则弹出
        r.encoding = r.apparent_encoding  # 将编码替换
        # return r.text  # 返回网页源代码
        # return r.content # 返回二进制文件
        return r.content.decode()  # 返回解码后的html文件

    except:
        return ''


def get_json(html_contain_json):
    """
    将含有json格式的html利用正则表达式提取出json
    :param html_contain_json:
    :return: json中需要的部分
    """
    rex = re.compile(r'\w+[(]{1}(.*)[)]{1}')  # 生成正则表达式对象
    res = rex.findall(html_contain_json)[0]
    json_datas = json.loads(res)  # 加载json文件
    return json_datas['data']


def save_as_csv(info_list):
    """
    保存方法1:  利用csv库保存为csv格式
    :param info_list: 需要保存的单个企业的所有stock信息
    :return: 无返回值
    """
    with open("./save/{}total_stock_3.csv".format(cur_time), 'a') as f:
        global flag  # 声明全局变量，只将列名称写进去一次

        w = csv.writer(f)
        if flag:
            w.writerow(['日期', '序号', '板块名称', '股票代码', '企业名称',
                        '最新价', '涨跌幅%', '涨跌额', '成交量(手)', '成交额(元)',
                        '振幅%', '最高', '最低', '今开', '昨收',
                        '量比', '换手率', '市盈率(动态)', '市净率', '总市值',
                        '流通市值', '60日涨跌幅', '年初至今涨跌幅', '涨速', '5分钟涨跌', '上市时间'])
            flag = False  # 只将列名称写进去一次
        else:
            w.writerow(info_list)  # 将传进来的列保存（单个企业的所有stock信息）


```

```
def main():
    start_time = time.time()  # 记录开始时间
    cur_time = time.strftime('%Y-%m-%d')  # 当前日期（年-月-日）
    # 记录所有板块的 json信息返回地址
    dict_board = {
        '上证A股': 'http://99.push2.eastmoney.com/api/qt/clist/get?cb=jQuery1124050018637467886 8_1617958044905&pn={}&pz=20&po=1&np=1&ut=bd1d9ddb04089700cf9c27f6f7426281&fltt=2&invt=2&fid=f3&fs=m:1+t:2,m:1+t:23&fields=f1,f2,f3,f4,f5,f6,f7,f8,f9,f10,f12,f13,f14,f15,f16,f17,f18,f20,f21,f23,f24,f25,f22,f11,f62,f128,f136,f115,f152&_=1617958044919',
        '沪深A股': 'http://31.push2.eastmoney.com/api/qt/clist/get?cb=jQuery112400347134466850884 7_1617964948682&pn={}&pz=20&po=1&np=1&ut=bd1d9ddb04089700cf9c27f6f7426281&fltt=2&invt=2&fid=f3&fs=m:0+t:6,m:0+t:80,m:1+t:2,m:1+t:23&fields=f1,f2,f3,f4,f5,f6,f7,f8,f9,f10,f12,f13,f14,f15,f16,f17,f18,f20,f21,f23,f24,f25,f22,f11,f62,f128,f136,f115,f152&_=1617964948730',
        '深证A股': 'http://31.push2.eastmoney.com/api/qt/clist/get?cb=jQuery112400347134466850884 7_1617964948684&pn={}&pz=20&po=1&np=1&ut=bd1d9ddb04089700cf9c27f6f7426281&fltt=2&invt=2&fid=f3&fs=m:0+t:6,m:0+t:80&fields=f1,f2,f3,f4,f5,f6,f7,f8,f9,f10,f12,f13,f14,f15,f16,f17,f18,f20,f21,f23,f24,f25,f22,f11,f62,f128,f136,f115,f152&_=1617964948741',
        '新股': 'http://31.push2.eastmoney.com/api/qt/clist/get?cb=jQuery112400347134466850884 7_1617964948684&pn={}&pz=20&po=1&np=1&ut=bd1d9ddb04089700cf9c27f6f7426281&fltt=2&invt=2&fid=f26&fs=m:0+f:8,m:1+f:8&fields=f1,f2,f3,f4,f5,f6,f7,f8,f9,f10,f12,f13,f14,f15,f16,f17,f18,f20,f21,f23,f24,f25,f26,f22,f11,f62,f128,f136,f115,f152&_=1617964948762',
        '创业板': 'http://31.push2.eastmoney.com/api/qt/clist/get?cb=jQuery112400347134466850884 7_1617964948684&pn={}&pz=20&po=1&np=1&ut=bd1d9ddb04089700cf9c27f6f7426281&fltt=2&invt=2&fid=f3&fs=m:0+t:80&fields=f1,f2,f3,f4,f5,f6,f7,f8,f9,f10,f12,f13,f14,f15,f16,f17,f18,f20,f21,f23,f24,f25,f22,f11,f62,f128,f136,f115,f152&_=1617964948788',
        '科创板': 'http://31.push2.eastmoney.com/api/qt/clist/get?cb=jQuery112400347134466850884 7_1617964948684&pn={}&pz=20&po=1&np=1&ut=bd1d9ddb04089700cf9c27f6f7426281&fltt=2&invt=2&fid=f3&fs=m:1+t:23&fields=f1,f2,f3,f4,f5,f6,f7,f8,f9,f10,f12,f13,f14,f15,f16,f17,f18,f20,f21,f23,f24,f25,f22,f11,f62,f128,f136,f115,f152&_=1617964948798',
        '沪股通': 'http://31.push2.eastmoney.com/api/qt/clist/get?cb=jQuery112400347134466850884 7_1617964948682&pn={}&pz=20&po=1&np=1&ut=bd1d9ddb04089700cf9c27f6f7426281&fltt=2&invt=2&fid=f26&fs=b:BK0707&fields=f1,f2,f3,f4,f5,f6,f7,f8,f9,f10,f12,f13,f14,f15,f16,f17,f18,f20,f21,f23,f24,f25,f26,f22,f11,f62,f128,f136,f115,f152&_=1617964948807',
        '深股通': 'http://31.push2.eastmoney.com/api/qt/clist/get?cb=jQuery112400347134466850884 7_1617964948684&pn={}&pz=20&po=1&np=1&ut=bd1d9ddb04089700cf9c27f6f7426281&fltt=2&invt=2&fid=f26&fs=b:BK0804&fields=f1,f2,f3,f4,f5,f6,f7,f8,f9,f10,f12,f13,f14,f15,f16,f17,f18,f20,f21,f23,f24,f25,f26,f22,f11,f62,f128,f136,f115,f152&_=1617964948827',
```

```
        'B股': 'http://31.push2.eastmoney.com/api/qt/clist/get?cb=jQuery112400347134466850884 7_1617964948684&pn={}&pz=20&po=1&np=1&ut=bd1d9ddb04089700cf9c27f6f7426281&fltt=2&invt=2&fid=f3&fs=m:0+t:7,m:1+t:3&fields=f1,f2,f3,f4,f5,f6,f7,f8,f9,f10,f12,f13,f14,f15,f16,f17,f18,f20,f21,f23,f24,f25,f22,f11,f62,f128,f136,f115,f152&_=1617964948836',
        '风险警示板': 'http://31.push2.eastmoney.com/api/qt/clist/get?cb=jQuery112400347134466850884 7_1617964948684&pn={}&pz=20&po=1&np=1&ut=bd1d9ddb04089700cf9c27f6f7426281&fltt=2&invt=2&fid=f3&fs=m:0+f:4,m:1+f:4&fields=f1,f2,f3,f4,f5,f6,f7,f8,f9,f10,f12,f13,f14,f15,f16,f17,f18,f20,f21,f23,f24,f25,f22,f11,f62,f128,f136,f115,f152&_=1617964948848'
    }

    board_total_stock_data = []  # 记录所有板块中的所有企业的stock信息
    data_count = 0  # 记录样本条数（每个企业的序号）
    for board in tqdm(dict_board.keys()):  # 用来循环每个板块
        board_name = board  # 板块名称
        board_base_url = dict_board[board]  # 板块的基础URL（不包含页数部分）
        index = 1  # 每个板块的页数从第一页开始

        while True:  # 用来循环该板块中的每个页面
            board_url = board_base_url.format(index)  # 每页的url（每页20个企业的股票信息）
            stock_html = get_html(board_url)  # 获取解码后的html
            json_datas = get_json(stock_html)  # 返回json文件
            board_stock_count = json_datas['total']  # 每个板块的企业总数量

            stock_total_data = json_datas['diff']  # 该页面中所有的20个企业信息保存在diff中
            for stock_data in stock_total_data:  # 循环每个企业信息
                data_count += 1
                stock_f12 = stock_data['f12']  # 股票代码
                stock_f14 = stock_data['f14']  # 企业名称
                stock_f2 = stock_data['f2']  # 最新价
                stock_f3 = str(stock_data['f3']) + '%'  # 涨跌幅%
                stock_f4 = stock_data['f4']  # 涨跌额
                stock_f5 = stock_data['f5']  # 成交量(手)
                stock_f6 = stock_data['f6']  # 成交额(元)
                stock_f7 = str(stock_data['f7']) + '%'  # 振幅%
                stock_f15 = stock_data['f15']  # 最高
                stock_f16 = stock_data['f16']  # 最低
                stock_f17 = stock_data['f17']  # 今开
                stock_f18 = stock_data['f18']  # 昨收
                stock_f10 = stock_data['f10']  # 量比
                stock_f8 = str(stock_data['f8']) + '%'  # 换手率
                stock_f9 = stock_data['f9']  # 市盈率(动态)
                stock_f23 = stock_data['f23']  # 市净率
                stock_f20 = stock_data['f20']  # 总市值
                stock_f21 = stock_data['f21']  # 流通市值
                stock_f24 = str(stock_data['f24']) + '%'  # 60日涨跌幅
                stock_f25 = str(stock_data['f25']) + '%'  # 年初至今涨跌幅
```

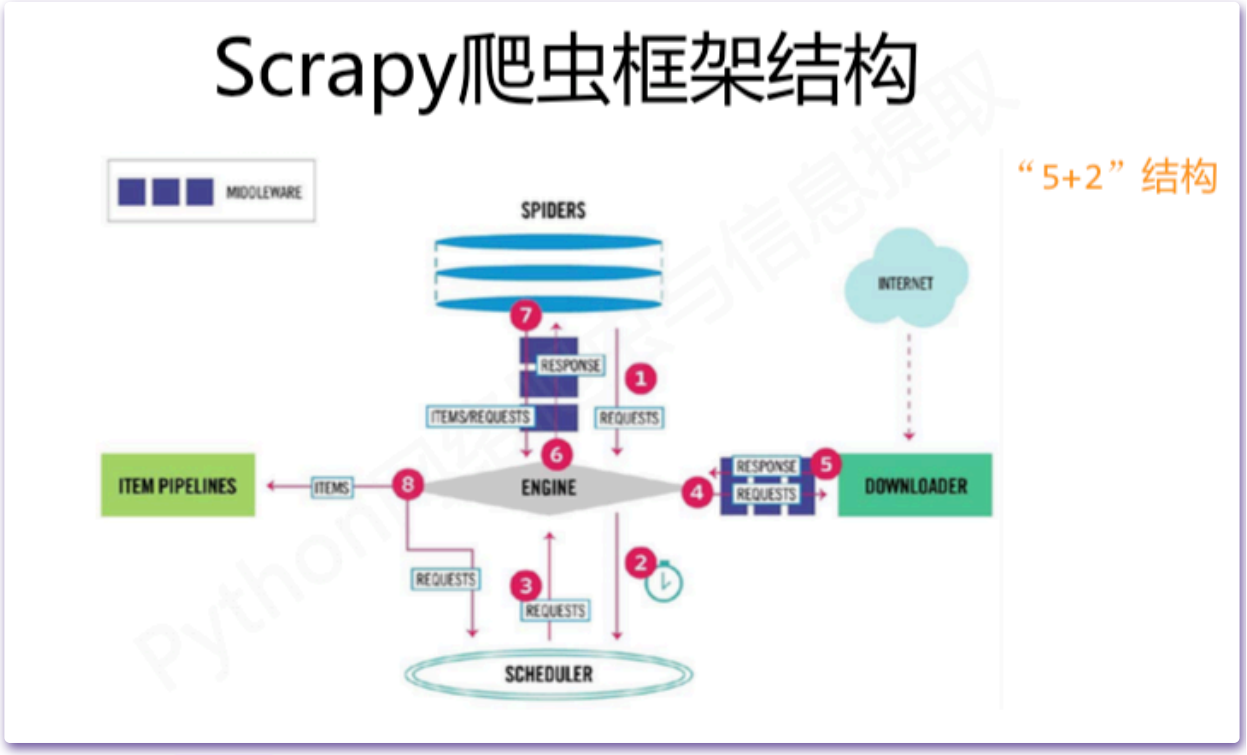
```
                stock_f22 = str(stock_data['f22']) + '%'  # 涨速
                stock_f11 = str(stock_data['f11']) + '%'  # 5分钟涨跌
                if board_name in ['新股', '沪股通', '深股通']:
                    stock_f26 = stock_data['f26']  # 上市时间（少数有上市时间）
                else:
                    stock_f26 = '空'

                # 该企业股票信息
                stock_information = [cur_time, data_count, board_name, stock_f12, stock_f14,
                                     stock_f2, stock_f3, stock_f4, stock_f5, stock_f6,
                                     stock_f7, stock_f15, stock_f16, stock_f17, stock_f18,
                                     stock_f10, stock_f8, stock_f9, stock_f23, stock_f20,
                                     stock_f21, stock_f24, stock_f25, stock_f22, stock_f11, stock_f26]

                board_total_stock_data.append(stock_information)  # 将该企业股票信息保存
                save_as_csv(stock_information)  # 写入csv文件

                # print(stock_information)
            index += 1

            if index == (board_stock_count // 20):  # 每个页面20个企业的信息，以此为停止条件
                break

    end_time = time.time()  # 记录结束时间

    print('用时：{} 秒，共解析网页：{} 个'.format(end_time - start_time, len(board_total_stock_data) // 20))
    # print(board_total_stock_data)
    print('数据信息', len(board_total_stock_data), '条')

    board_total_stock_data = pd.DataFrame(board_total_stock_data)
    board_total_stock_data.columns = ['日期', '序号', '板块名称', '股票代码', '企业名称',
                                      '最新价', '涨跌幅%', '涨跌额', '成交量(手)', '成交额(元)',
                                      '振幅%', '最高', '最低', '今开', '昨收',
                                      '量比', '换手率', '市盈率(动态)', '市净率', '总市值',
                                      '流通市值', '60日涨跌幅', '年初至今涨跌幅', '涨速', '5分钟涨跌', '上市时间']
    # 保存方法2： pandas保存为excel格式

    board_total_stock_data.to_excel('./save/{}total_stock_3.xlsx'.format(cur_time), index=None)

```

```python

if __name__ == '__main__':
    main()



"""运行结果"""
100%|██████████| 10/10 [00:49<00:00,  2.28s/it]
用时："""49.348698139190674 秒"""，共解析网页："""602""" 个
数据信息 """602""" 条
```

5 第四章 网络爬虫之框架

5.1 scrapy框架的简介

- scrapy 爬虫框架结构：



1. Engine

   (1) 控制所有模块之间的数据流
   (2) 根据条件触发事件

   不需要用户更改

2. Downloader

   根据请求下载网页

   不需要用户更改

3. Scheduler

   对所有爬取请求进行调度管理

   不需要用户更改

4. Downloader Middleware

   (1) **目的:** 实施Engine、Scheduler和Downloader 之间进行用户可配置的控制

   (2) **功能:** 修改、丢弃、新增请求或响应

   用户可以编写 配置代码

5. Spider

   (1) 解析Downloader返回的响应(Response)

(2) 产生爬取项(scraped item)
(3) 产生额外的爬取请求(Request)

需要用户编写配置代码

6. `Item Pipelines`

(1) 以流水线方式处理Spider产生的爬取项
(2) 由一组操作顺序组成，类似流水线，每个操作是一个Item Pipelines类型

(3) 可能操作包括:清理、检验和查重爬取项中的HTML数据、将数据存储到数据库

需要用户编写配置代码

7. `Spider Middleware`

(1) **目的:**对请求和爬取项的再处理

(2) **功能:**修改、丢弃、新增请求或爬取项

- `Requests 和 Scrapy 比较`
  - 相同点:
    1. 两者都可以进行页面请求和爬取，Python爬虫的两个重要技术路线
    2. 两者可用性都好，文档丰富，入门简单
    3. 两者都没有处理js、提交表单、应对验证码等功能(可扩展)

requests vs. Scrapy

页面级爬虫	网站级爬虫
功能库	框架
并发性考虑不足，性能较差	并发性好，性能较高
重点在于页面下载	重点在于爬虫结构
定制灵活	一般定制灵活，深度定制困难
上手十分简单	入门稍难

- Scrapy `常用命令`

命令	说明	格式
startproject	创建一个新工程	scrapy startproject <name> [dir]
genspider	创建一个爬虫	scrapy genspider [options] <name> <domain>
settings	获得爬虫配置信息	scrapy settings [options]
crawl	运行一个爬虫	scrapy crawl <spider>
list	列出工程中所有爬虫	scrapy list
shell	启动URL调试命令行	scrapy shell [url]

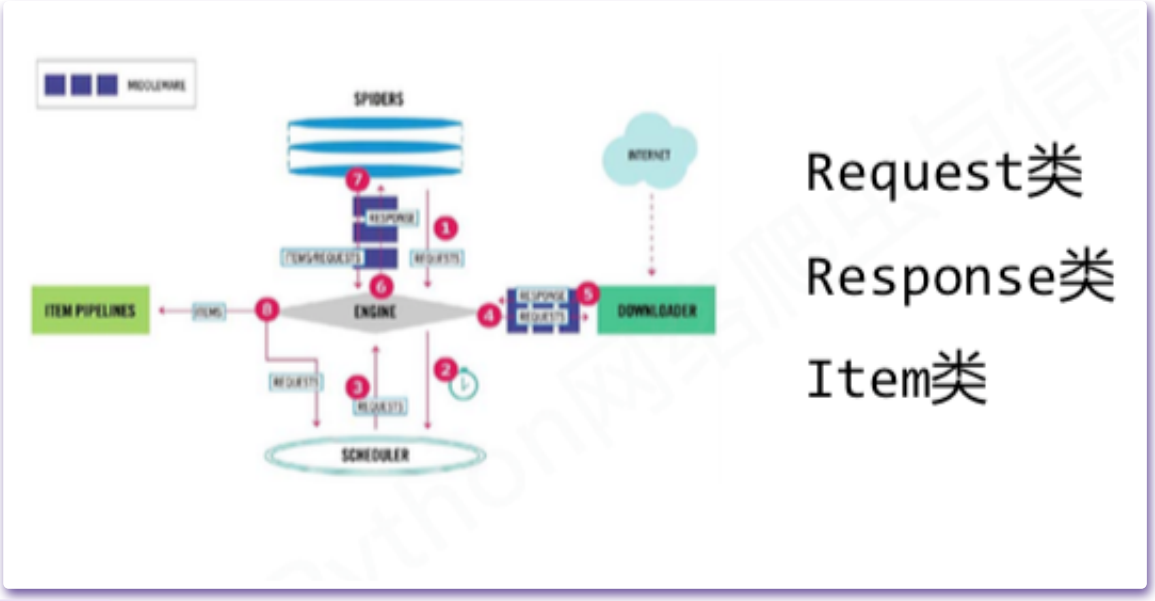
- 生成d额工程目录

```
python123demo/          ──→  外层目录
    scrapy.cfg          ──→  部署Scrapy爬虫的配置文件
    python123demo/      ──→  Scrapy框架的用户自定义Python代码
        __init__.py     ──→  初始化脚本
        items.py        ──→  Items代码模板（继承类）
        middlewares.py  ──→  Middlewares代码模板（继承类）
        pipelines.py    ──→  Pipelines代码模板（继承类）
        settings.py     ──→  Scrapy爬虫的配置文件
        [spiders/]      ──→  Spiders代码模板目录（继承类）
目录结构  __pycache__/  ──→  缓存目录，无需修改
```

```
spiders/        ──→  Spiders代码模板目录（继承类）
    __init__.py ──→  初始文件，无需修改
    __pycache__/ ──→ 缓存目录，无需修改

内层目录结构        用户自定义的spider代码增加在此处
```

5.2 scrapy框架的数据类型

- scrapy爬虫的数据类型


SPIDERS
INTERNET
ITEM PIPELINES
ENGINE
DOWNLOADER
SCHEDULER
Request类
Response类
Item类

- Request类：

Request类
class scrapy.http.Request()
Request对象表示一个HTTP请求
由Spider生成，由Downloader执行

Request类

属性或方法	说明
.url	Request对应的请求URL地址
.method	对应的请求方法，'GET' 'POST'等
.headers	字典类型风格的请求头
.body	请求内容主体，字符串类型
.meta	用户添加的扩展信息，在Scrapy内部模块间传递信息使用
.copy()	复制该请求

- Response类：

Response类

class scrapy.http.Response()

Response对象表示一个HTTP响应

由Downloader生成，由Spider处理

Response类型

属性或方法	说明
.url	Response对应的URL地址
.status	HTTP状态码，默认是200
.headers	Response对应的头部信息
.body	Response对应的内容信息，字符串类型
.flags	一组标记
.request	产生Response类型对应的Request对象
.copy()	复制该响应

- Item类：

Item类

class scrapy.item.Item()

Item对象表示一个从HTML页面中提取的信息内容

由Spider生成，由Item Pipeline处理

Item类似字典类型，可以按照字典类型操作

5.3 scrapy爬虫提取信息的方法

- 提取信息方法

Scrapy爬虫支持多种HTML信息提取方法：

- Beautiful Soup
- lxml
- re
- XPath Selector
- CSS Selector

CSS Selector的基本使用

```
<HTML>.css('a::attr(href)').extract()
```

标签名称　　标签属性

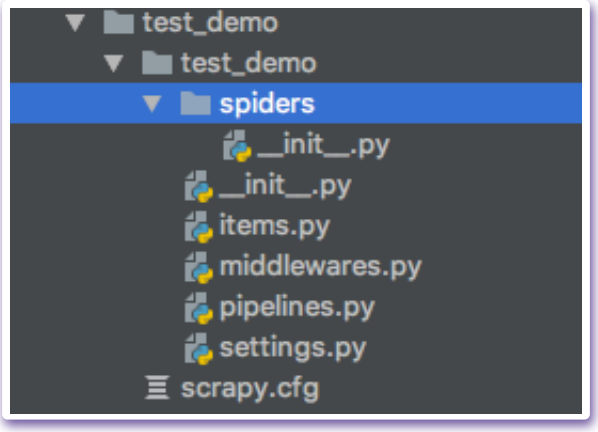
CSS Selector由W3C组织维护并规范

5.4 scrapy小例子：爬取网页html并保存

- 步骤一：建立一个Scrapy爬虫工程

```
mengzhichaodeMacBook-Pro:第四周 mengzhichao$ scrapy startproject test_demo
New Scrapy project 'test_demo', using template directory '/Users/mengzhichao/anaconda3/lib/python3.7
/site-packages/scrapy/templates/project', created in:
    /Users/mengzhichao/Desktop/文件/16、python代码/Sample_Code/9、爬虫/9.2_北理工爬虫/第四周/test_de
mo

You can start your first spider with:
    cd test_demo
    scrapy genspider example example.com
mengzhichaodeMacBook-Pro:第四周 mengzhichao$
```

生成的目录结构：



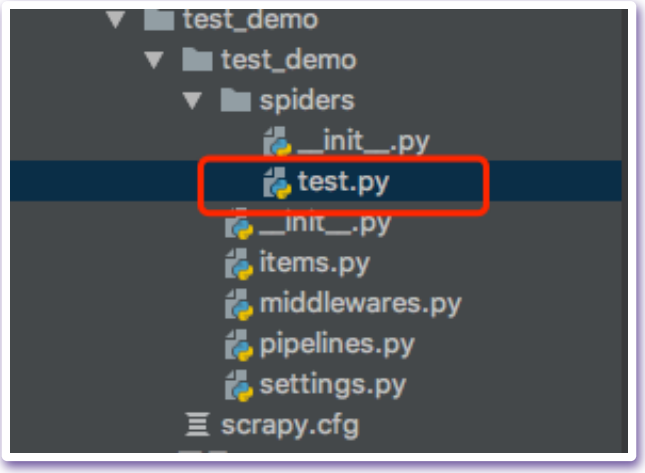
- 步骤二：在工程中产生一个Scrapy爬虫

该命令作用：

1. 生成一个名称为test的Spider
2. 在spiders目录下增加代码文件demo.py
3. 该命令仅用于生成demo.py，该文件也可以手工生成

```
mengzhichaodeMacBook-Pro:第四周 mengzhichao$ cd test_demo/
mengzhichaodeMacBook-Pro:test_demo mengzhichao$ scrapy genspider test 111
Created spider 'test' using template 'basic' in module:
  test_demo.spiders.test
mengzhichaodeMacBook-Pro:test_demo mengzhichao$
```

当前目录结构：



Test.py文件

```python
import scrapy


class TestSpider(scrapy.Spider):
    name = 'test'
    allowed_domains = ['111']
    start_urls = ['http://111/']

    def parse(self, response):
        pass
```

- 步骤三：配置产生的spider爬虫：

  配置：

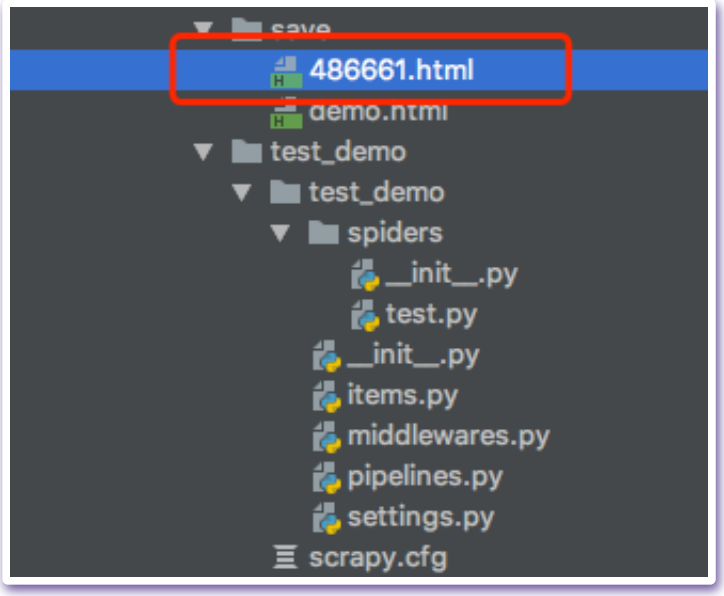
  1. 初始化url地址
  2. 获取页面后的解析方式

```python
import scrapy


class TestSpider(scrapy.Spider):
    name = 'test'
    # allowed_domains = ['111'] # 可选
    start_urls = ['http://www.pc6.com/mac/486661.html']

    def parse(self, response):
        fname = response.url.split('/')[-1]
        with open('../save/{}'.format(fname), 'wb') as f:
            f.write(response.body)
        self.log('Saved file %s.' % fname)
```

```python
import scrapy


class TestSpider(scrapy.Spider):
    name = 'test'
    # allowed_domains = ['111'] # 可选
    start_urls = ['http://www.pc6.com/mac/486661.hratml']

    def parse(self, response):
        fname = response.url.split('/')[-1]
        with open('../save/{}'.format(fname), 'wb') as f:
            f.write(response.body)
        self.log('Saved file %s.' % fname)

```

- 步骤四：运行爬虫，获取页面

```
mengzhichaodeMacBook-Pro:test_demo mengzhichao$ scrapy crawl test
2021-04-15 11:24:50 [scrapy.utils.log] INFO: Scrapy 2.5.0 started (bot: test_de
```


save
486661.html
demo.html
test_demo
test_demo
spiders
__init__.py
test.py
__init__.py
items.py
middlewares.py
pipelines.py
settings.py
scrapy.cfg